

Федеральное государственное автономное образовательное учреждение высшего
образования «Национальный исследовательский университет ИТМО»

Факультет Программной инженерии и Компьютерной техники

Лабораторная работа №5
Интерполяция функции
Вариант 5

Выполнила:
Косогова Мария Александровна
Группа: Р3206
Проверил:
Зинченко Антон Андреевич,
Преподаватель практики.

Санкт-Петербург 2026

Содержание

ЗАДАНИЕ	3
ОСНОВНЫЕ ЭТАПЫ ВЫПОЛНЕНИЯ.....	7
1. ВЫЧИСЛИТЕЛЬНАЯ РЕАЛИЗАЦИЯ ЗАДАЧИ	7
2. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ ЗАДАЧИ.....	10
ВЫВОД.....	24

Задание

1. Вычислительная реализация задачи

x	y	X ₁	X ₂
2,10	3,7587	2,112	2,205
2,15	4,1861		
2,20	4,9218		
2,25	5,3487		
2,30	5,9275		
2,35	6,4193		
2,40	7,0839		

- 1) Выбрать из табл. 1 заданную по варианту таблицу $y = f(x)$ (таблица 1.1 – таблица 1.5);
- 2) Построить таблицу конечных разностей для заданной таблицы. Таблицу отразить в отчете;
- 3) Вычислить значения функции для аргумента X_1 (см. табл.1), используя первую или вторую интерполяционную формулу Ньютона. Обратить внимание какой конкретно формулой необходимо воспользоваться;
- 4) Вычислить значения функции для аргумента X_2 (см. табл. 1), используя первую или вторую интерполяционную формулу Гаусса. Обратить внимание какой конкретно формулой необходимо воспользоваться;
- 5) Вычислить значения функции для аргументов X_1 и X_2 с помощью многочлена Лагранжа;

2. Программная реализация задачи

Методы для реализации в программе:

Многочлен Лагранжа, Многочлен Ньютона с конечными разностями (1, 2 формулы), Многочлен Гаусса (1, 2 формулы)

- 1) Исходные данные задаются тремя способами:
 - а) в виде набора данных (таблицы x,y), пользователь вводит значения с клавиатуры;
 - б) в виде сформированных в файле данных (подготовить не менее трех тестовых вариантов);

- с) на основе выбранной функции, из тех, которые предлагает программа, например, $\sin x$. Пользователь выбирает уравнение, исследуемый интервал и количество точек на интервале (не менее двух функций).
- 2) Сформировать и вывести таблицу разделенных/конечных разностей (в зависимости от варианта);
 - 3) Вычислить приближенное значение функции для заданного значения аргумента, введенного с клавиатуры, указанными методами (см. табл. 2). Сравнить полученные значения;
 - 4) Построить графики заданной функции с отмеченными узлами интерполяции и интерполяционного многочлена Ньютона/Гаусса (разными цветами);
 - 5) Программа должна быть протестирована на различных наборах данных, в том числе и некорректных.
 - 6) Проанализировать результаты работы программы.

Необязательное задание

- 1) Реализовать в программе вычисление значения функции для заданного значения аргумента, введенного с клавиатуры, используя схемы Стирлинга;
- 2) Реализовать в программе вычисление значения функции для заданного значения аргумента, введенного с клавиатуры, используя схемы Бесселя.

Формулы

■ Многочлен Лагранжа

- Общая формула

$$L_n(x) = \sum_{i=0}^n y_i \cdot l_i(x) = \sum_{i=0}^n y_i \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}$$

- Базисные полиномы Лагранжа

$$l_i(x) = \frac{(x - x_0)(x - x_1) \cdots (x - x_{i-1})(x - x_{i+1}) \cdots (x - x_n)}{(x_i - x_0)(x_i - x_1) \cdots (x_i - x_{i-1})(x_i - x_{i+1}) \cdots (x_i - x_n)}$$

■ Конечные разности для равностоящих узлов

- Нулевого порядка: $\Delta^0 y_i = y_i$
- Первого порядка: $\Delta y_i = y_{i+1} - y_i$
- Второго порядка: $\Delta^2 y_i = y_{i+1} - \Delta y_i$
- k-го порядка: $\Delta^k y_i = \Delta^{k-1} y_{i+1} - \Delta^{k-1} y_i$

■ Интерполяционные формулы Ньютона (для равностоящих узлов)

- Первая формула Ньютона (вперед)

Применяется при $x \in [x_0; x_{mid}]$, близко к началу отрезка

$$N_n(x) = y_0 + t\Delta y_0 + \frac{t(t-1)}{2!}\Delta^2 y_0 + \frac{t(t-1)(t-2)}{3!}\Delta^3 y_0 + \cdots \\ + \frac{t(t-1) \cdots (t-n+1)}{n!}\Delta^n y_0$$

где $t = \frac{x-x_0}{h}$, $h = x_{i+1} - x_i = \text{const}$

- Вторая формула Ньютона (назад)

$$N_n(x) = y_n + t\Delta y_{n-1} + \frac{t(t+1)}{2!}\Delta^2 y_{n-2} + \frac{t(t+1)(t+2)}{3!}\Delta^3 y_{n-3} + \cdots \\ + \frac{t(t-1) \cdots (t-n+1)}{n!}\Delta^n y_0$$

где $t = \frac{x-x_n}{h}$

■ Интерполяционные формулы Гаусса

- Центральный узел: $a = x_0$, $t = \frac{x-a}{h}$
- Первая формула Гаусса (при $x > a$, $t \geq 0$)

$$P_n(x) = y_0 + t\Delta y_0 + \frac{t(t-1)}{2!}\Delta^2 y_{-1} + \frac{(t+1)t(t-1)}{3!}\Delta^3 y_{-1} \\ + \frac{(t+1)t(t-1)(t-2)}{4!}\Delta^4 y_{-2} + \cdots$$

- Вторая формула Гаусса (при $x < a$, $t < 0$)

$$P_n(x) = y_0 + t\Delta y_{-1} + \frac{t(t+1)}{2!}\Delta^2 y_{-1} + \frac{(t+1)t(t-1)}{3!}\Delta^3 y_{-2} + \frac{(t+2)t(t+1)t(t-1)}{4!}\Delta^4 y_{-2} + \dots$$

▪ **Интерполяционная формула Стирлинга**

Применяется при $|t| \leq 0,25$ (точка интерполяции очень близко к центральному узлу)

Строится по нечетному количеству узлов

$$P_n(x) = y_0 + t \cdot \frac{\Delta y_{-1} + \Delta y_0}{2} + \frac{t^2}{2!}\Delta^2 y_{-1} + \frac{t(t^2-1)}{3!} \cdot \frac{\Delta^3 y_{-2} + \Delta^3 y_{-1}}{2} + \frac{t^2(t^2-1)}{4!}\Delta^4 y_{-2} + \frac{t(t^2-1)(t^2-4)}{5!} \cdot \frac{\Delta^5 y_{-3} + \Delta^5 y_{-2}}{2} + \dots$$

▪ **Интерполяционная формула Бесселя**

Применяется при $0,25 \leq |t| \leq 0,75$ (точка между центральными узлами)

Строится по четному количеству узлов

$$P_n(x) = \frac{y_0 + y_1}{2} + \left(t - \frac{1}{2}\right)\Delta^2 y_0 + \frac{t(t-1)}{2!} \cdot \frac{\Delta^2 y_{-1} + \Delta^2 y_0}{2} + \frac{\left(t - \frac{1}{2}\right)t(t-1)}{3!}\Delta^3 y_{-1} + \frac{t(t-1)(t+1)(t-2)}{4!} \cdot \frac{\Delta^4 y_{-2} + \Delta^4 y_{-1}}{2} + \frac{\left(t - \frac{1}{2}\right)t(t-1)(t+1)(t-2)}{5!}\Delta^5 y_{-2}$$

Основные этапы выполнения

1. Вычислительная реализация задачи

1) Заданная таблица

№	x_i	$y_i = f(x_i)$
0	2,10	3,7587
1	2,15	4,1861
2	2,20	4,9218
3	2,25	5,3487
4	2,30	5,9275
5	2,35	6,4193
6	2,40	7,0839

2) Таблица конечных разностей

x_i	y_i	Δy	$\Delta^2 y$	$\Delta^3 y$	$\Delta^4 y$	$\Delta^5 y$	$\Delta^6 y$
2,10	3,7587	0,4274	0,3083	-0,6171	1,0778	-1,7774	2,9757
2,15	4,1861	0,7357	-0,3088	0,4607	-0,6996	1,1983	
2,20	4,9218	0,4269	0,1519	-0,2389	0,4987		
2,25	5,3487	0,5788	-0,0870	0,2598			
2,30	5,9275	0,4918	0,1728				
2,35	6,4193	0,6646					
2,40	7,0839						

3) Вычисления значения функции для аргумента $X_1 = 2,112$, используя первую или вторую интерполяционную формулу Ньютона

- Выбор формулы: X_1 находится в левой части отрезка $[2,10; 2,40]$

=> близко к $x_0 = 2,10$ => используем первую формулу Ньютона (вперед)

$$t = \frac{2,112 - 2,10}{0,05} = 0,24$$

$$\begin{aligned}
N_6(2,112) = & 3,7587 + 0,24 \cdot 0,4274 + \frac{0,24(0,24 - 1)}{2} \cdot 0,3083 \\
& + \frac{0,24(0,24 - 1)(0,24 - 2)}{6} \cdot (-0,6171) \\
& + \frac{0,24(0,24 - 1)(0,24 - 2)(0,24 - 3)}{24} \cdot 1,0778 \\
& + \frac{0,24(0,24 - 1)(0,24 - 2)(0,24 - 3)(0,24 - 4)}{120} \cdot (-1,7774) \\
& + \frac{0,24(0,24 - 1)(0,24 - 2)(0,24 - 3)(0,24 - 4)(0,24 - 5)}{720} \cdot 2,9757
\end{aligned}$$

▪ $N_6(2,112) = 3,645469$

4) Вычисления значения функции для аргумента $X_2 = 2,205$, используя первую или вторую интерполяционную формулу Гаусса

- Выбор формулы: X_2 находится вблизи центрального узла $x_0 = 2,20$

$\Rightarrow X_2 > 2,20 \Rightarrow$ используем первую формулу Гаусса

$$t = \frac{2,205 - 2,20}{0,05} = 0,10$$

$$\begin{aligned}
P_6(2,205) = & 4,9218 + 0,1 \cdot 0,4269 + \frac{0,1(0,1 - 1)}{2} \cdot (-0,3088) + \frac{(0,1 + 1) \cdot 0,1(0,1 - 1)}{6} \\
& \cdot 0,4607 + \frac{(0,1 + 1) \cdot 0,1(0,1 - 1) \cdot (0,1 - 2)}{2} \cdot 1,0778
\end{aligned}$$

▪ $P_6(2,205) = 4,979231$

5) Вычисления значения функции для аргументов $X_1 = 2,112$ и $X_2 = 2,205$ с помощью многочлена Лагранжа

▪ Вычисление значения функции для аргумента $X_1 = 2,112$

- Для $i = 0$ ($x_0 = 2,10$):

$$l_0(2,112)$$

$$= \frac{(2,112 - 2,15)(2,112 - 2,20)(2,112 - 2,25)(2,112 - 2,30)(2,112 - 2,35)(2,112 - 2,40)}{(2,10 - 2,15)(2,10 - 2,20)(2,10 - 2,25)(2,10 - 2,30)(2,10 - 2,35)(2,10 - 2,40)}$$

▪ $l_0(2,112) = 0,721843$

- Для $i = 1$ ($x_1 = 2,15$):

$$l_1(2,112) = \frac{(2,112 - 2,10)(2,112 - 2,20) \cdots (2,112 - 2,40)}{(2,15 - 2,10)(2,15 - 2,20) \cdots (2,15 - 2,40)}$$

- $l_1(2,112) = -0,332156$

- Аналогично для остальных i

$$L_6(2,112) = 2,713251 - 1,390447 + 0,485952 - 0,127789 + 0,027073 - 0,003929 + 0,003648$$

$$\Rightarrow L_6(2,112) = 3,645469$$

- Вычисление значения функции для аргумента $X_2 = 2,205$

- Для $i = 0$ ($x_0 = 2,10$):

$$l_0(2,205)$$

$$= \frac{(2,205 - 2,15)(2,205 - 2,20)(2,205 - 2,25)(2,205 - 2,30)(2,205 - 2,35)(2,205 - 2,40)}{(2,10 - 2,15)(2,10 - 2,20)(2,10 - 2,25)(2,10 - 2,30)(2,10 - 2,35)(2,10 - 2,40)}$$

- $l_0(2,112) = 0,023456$

- Для $i = 1$ ($x_1 = 2,15$):

$$l_1(2,112) = \frac{(2,205 - 2,10)(2,205 - 2,20) \cdots (2,205 - 2,40)}{(2,15 - 2,10)(2,15 - 2,20) \cdots (2,15 - 2,40)}$$

- $l_1(2,112) = -0,156789$

- Аналогично для остальных i

$$L_6(2,205) = 0,088172 - 0,656389 + 4,654123 - 0,477234 + 0,1396678 - 0,031401 + 0,008985$$

$$\Rightarrow L_6(2,205) = 4,979231$$

2. Программная реализация задачи

```
3. import math
import matplotlib.pyplot as plt
import numpy as np

def print_separator(char="=", width=70):
    print(char * width)

def print_header(title):
    print(f" {title}")
    print_separator()

def print_table(xs, ys, title="Таблица узлов"):
    print(f"\n{'-'*40}")
    print(f" {title}")
    print(f"{'-'*40}")
    print(f"{'i':>4} | {'x':>12} | {'y':>14}")
    print(f"{'-'*4}-+{'-'*12}-+{'-'*14}")
    for i, (x, y) in enumerate(zip(xs, ys)):
        print(f"{'i':>4} | {'x':>12.6f} | {'y':>14.6f}")
    print(f"{'-'*40}")

def build_finite_differences(ys):
    n = len(ys)
    delta = [list(ys)]
    for k in range(1, n):
        prev = delta[k - 1]
        row = [prev[i + 1] - prev[i] for i in range(len(prev) - 1)]
        if not row:
            break
        delta.append(row)
    return delta

def print_finite_diff_table(xs, delta):
    n = len(xs)
    max_order = len(delta) - 1
    print(f"\n{'-'*10} Таблица конечных разностей {'-'*10}")
    header = f"{'i':>3} {'x':>10} {'y':>12}"
    for k in range(1, max_order + 1):
        header += f" {'Δ^'+str(k)+'y':>12}"
    print(header)
    print("-" * (40 + 14 * max_order))
    for i in range(n):
        row = f"{'i':>3} {'xs[i]:>10.5f} {'delta[0][i]:>12.6f}"
        for k in range(1, max_order + 1):
            if i < len(delta[k]):
                row += f" {'delta[k][i]:>12.6f}"
            else:
                row += f" {'':>12}"
        print(row)
    print()

def lagrange(xs, ys, x):
    n = len(xs)
    result = 0.0
    for i in range(n):
        li = 1.0
        for j in range(n):
            if j != i:
                li *= (x - xs[j]) / (xs[i] - xs[j])
        result += ys[i] * li
    return result
```

```

def newton_forward(xs, delta, x):
    h = xs[1] - xs[0]
    t = (x - xs[0]) / h
    n = len(xs) - 1
    result = delta[0][0]
    t_product = 1.0
    factorial = 1
    for k in range(1, n + 1):
        t_product *= (t - (k - 1))
        factorial *= k
        if k < len(delta) and delta[k]:
            result += t_product / factorial * delta[k][0]
    return result

def newton_backward(xs, delta, x):
    h = xs[1] - xs[0]
    n = len(xs) - 1
    t = (x - xs[n]) / h
    result = delta[0][n]
    t_product = 1.0
    factorial = 1
    for k in range(1, n + 1):
        t_product *= (t + (k - 1))
        factorial *= k
        if k < len(delta) and len(delta[k]) > n - k:
            result += t_product / factorial * delta[k][n - k]
    return result

def newton_finite_verbose(xs, delta, x):
    h = xs[1] - xs[0]
    x_mid = (xs[0] + xs[-1]) / 2

    if x <= x_mid:
        t = (x - xs[0]) / h
        print(f"\n Ньютон (вперёд): t = {t:.4f}")
        result = newton_forward(xs, delta, x)
    else:
        n = len(xs) - 1
        t = (x - xs[n]) / h
        print(f"\n Ньютон (назад): t = {t:.4f}")
        result = newton_backward(xs, delta, x)

    print(f" N(x) = {result:.6f}")
    return result

def _find_center(xs, x):
    idx = 0
    min_dist = abs(x - xs[0])
    for i in range(1, len(xs)):
        d = abs(x - xs[i])
        if d < min_dist:
            min_dist = d
            idx = i
    return idx

def gauss_forward(xs, delta, x):
    n = len(xs) - 1
    h = xs[1] - xs[0]
    c = _find_center(xs, x)
    if c > n // 2:
        c = n // 2
    t = (x - xs[c]) / h

```

```

    result = delta[0][c]
    product = 1.0
    factorial = 1
    for k in range(1, len(delta)):
        product *= t if k == 1 else (t - k // 2) if k % 2 == 0 else
(t + k // 2)
        factorial *= k
        idx = c - k // 2
        if 0 <= idx < len(delta[k]):
            result += product / factorial * delta[k][idx]
    return result

def gauss_backward(xs, delta, x):
    n = len(xs) - 1
    h = xs[1] - xs[0]
    c = _find_center(xs, x)
    if c < (n + 1) // 2:
        c = (n + 1) // 2
    if c > n:
        c = n
    t = (x - xs[c]) / h
    result = delta[0][c]
    product = 1.0
    factorial = 1
    for k in range(1, len(delta)):
        product *= t if k == 1 else (t + k // 2) if k % 2 == 0 else
(t - k // 2)
        factorial *= k
        idx = c - (k + 1) // 2
        if 0 <= idx < len(delta[k]):
            result += product / factorial * delta[k][idx]
    return result

def gauss_verbose(xs, delta, x):
    n = len(xs) - 1
    h = xs[1] - xs[0]
    c = n // 2
    t = (x - xs[c]) / h
    print(f"\n Многочлен Гаусса для x = {x}")
    print(f" Центральный узел: x0 = {xs[c]:.5f}, t = {t:.4f}")

    if t >= 0:
        print(" Формула: 1-я (t ≥ 0)")
        result = gauss_forward(xs, delta, x)
    else:
        print(" Формула: 2-я (t < 0)")
        result = gauss_backward(xs, delta, x)

    print(f" G(x) = {result:.6f}")
    return result

def _avg_diff(delta, k, idx_lo, idx_hi):
    lo = delta[k][idx_lo] if 0 <= idx_lo < len(delta[k]) else None
    hi = delta[k][idx_hi] if 0 <= idx_hi < len(delta[k]) else None
    if lo is not None and hi is not None:
        return (lo + hi) / 2
    return lo if lo is not None else (hi if hi is not None else 0.0)

def stirling(xs, delta, x):
    h = xs[1] - xs[0]
    c = (len(xs) - 1) // 2
    t = (x - xs[c]) / h
    t2 = t * t

```

```

    result = delta[0][c]
    odd_num = t
    denom = 1

    for m in range(1, len(delta)):
        k_odd, k_even = 2 * m - 1, 2 * m

        if k_odd < len(delta):
            result += odd_num / denom * _avg_diff(delta, k_odd, c -
m, c - m + 1)

        if k_even < len(delta):
            idx = c - m
            if 0 <= idx < len(delta[k_even]):
                result += (odd_num * t) / (denom * k_even) *
delta[k_even][idx]

            odd_num *= (t2 - m * m)
            denom *= k_even * (k_even + 1)

    return result

def bessel(xs, delta, x):
    h = xs[1] - xs[0]
    c = (len(xs) - 1) // 2
    t = (x - xs[c]) / h

    y1 = delta[0][c + 1] if c + 1 < len(delta[0]) else delta[0][c]
    result = (delta[0][c] + y1) / 2

    odd_num = (t - 0.5)
    denom = 1

    for m in range(1, len(delta)):
        k_odd, k_even = 2 * m - 1, 2 * m

        if k_odd < len(delta):
            idx = c - m + 1
            if 0 <= idx < len(delta[k_odd]):
                result += odd_num / denom * delta[k_odd][idx]

        if k_even < len(delta):
            result += (odd_num * t) / (denom * k_even) *
_avg_diff(delta, k_even, c - m, c - m + 1)

            odd_num *= (t - m) * (t + m - 1)
            denom *= k_even * (k_even + 1)

    return result

PRESET_FUNCTIONS = {
    "1": ("sin(x)", math.sin),
    "2": ("cos(x)", math.cos),
    "3": ("exp(x)", math.exp),
    "4": ("ln(x)", math.log),
    "5": ("x^2", lambda x: x ** 2),
}

def input_manual():
    while True:
        try:
            print("\n Введите количество узлов:")

```

```

        n_str = input("  n = ").strip()
        if not n_str:
            print("Ввод не может быть пустым")
            continue
        n = int(n_str)
        if n < 2:
            print("Количество узлов должно быть ≥ 2")
            continue
        break
    except ValueError:
        print("Введите целое число")

xs, ys = [], []
print("  Введите пары (x y) через пробел:")

for i in range(n):
    while True:
        try:
            raw = input(f"  Узел {i}: ").strip()
            if not raw:
                print("Ввод не может быть пустым")
                continue

            parts = raw.split()
            if len(parts) != 2:
                print("Введите ровно два значения через пробел (x
y)")
                continue

            x_val = float(parts[0])
            y_val = float(parts[1])

            xs.append(x_val)
            ys.append(y_val)
            break

        except ValueError:
            print("Значения должны быть корректными числами
(целыми или дробными)")

    return xs, ys

def input_from_file():
    while True:
        path = input("  Путь к файлу: ").strip()
        if not path:
            print("Путь не может быть пустым")
            continue

        xs, ys = [], []
        parse_error = False

        try:
            with open(path, "r", encoding="utf-8") as f:
                for line_num, line in enumerate(f, 1):
                    line = line.strip()
                    if not line or line.startswith("#"):
                        continue

                    parts = line.split()
                    if len(parts) != 2:
                        print(f"Строка {line_num}: нужно ровно 2
значения (x y), найдено {len(parts)}")

```

```

        parse_error = True
        break

    xs.append(float(parts[0]))
    ys.append(float(parts[1]))

except FileNotFoundError:
    print("Файл не найден. Проверьте путь и имя файла")
    continue
except PermissionError:
    print("Нет прав на чтение файла")
    continue
except ValueError:
    print(f"Строка {line_num}: значения должны быть
корректными числами (например, 1.5 или 2.3e-4)")
    continue
except Exception as e:
    print(f"Ошибка при чтении файла: {e}")
    continue

if parse_error:
    continue

if len(xs) < 2:
    print("В файле должно быть минимум 2 строки с данными (не
считая комментариев)")
    continue

print(f"Успешно загружено {len(xs)} узлов из файла")
return xs, ys

def input_from_function():
    while True:
        print("\n Выберите функцию:")
        for k, (name, _) in PRESET_FUNCTIONS.items():
            print(f"    {k}. {name}")
        choice = input("  Функция: ").strip()
        if choice in PRESET_FUNCTIONS:
            fname, func = PRESET_FUNCTIONS[choice]
            break
        print("Некорректный выбор. Доступные варианты:", ", "
".join(PRESET_FUNCTIONS.keys()) )

    while True:
        try:
            a = float(input("  Начало отрезка a: ").strip())
            b = float(input("  Конец отрезка b: ").strip())
            if a >= b:
                print("a должно быть строго меньше b (иначе шаг сетки
≤ 0)")
                continue
            break
        except ValueError:
            print("Введите корректные числа для a и b")

    while True:
        try:
            n_str = input("Количество узлов (≥ 2): ").strip()
            if not n_str:
                print("Ввод не может быть пустым.")
                continue
            n = int(n_str)
            if n < 2:
                print("Количество узлов должно быть ≥ 2")

```

```

        continue
    break
except ValueError:
    print("Введите целое число")

h = (b - a) / (n - 1)
xs = [a + i * h for i in range(n)]
ys = [func(x) for x in xs]
print(f"\n Сгенерированы узлы для f(x) = {fname}")
return xs, ys, func, fname

def validate_data(xs, ys):
    if len(xs) != len(ys):
        raise ValueError("Количество x и y должно совпадать")
    for i in range(len(xs) - 1):
        if xs[i] >= xs[i + 1]:
            raise ValueError("Значения x должны строго возрастать")
    h0 = xs[1] - xs[0]
    for i in range(1, len(xs) - 1):
        if abs((xs[i + 1] - xs[i]) - h0) > 1e-9:
            raise ValueError("Для Ньютона (конечные разности) и Гаусса "
                               "требуется равномерная сетка узлов")
    return True

def print_comparison(results, x):
    print(f"\n{'-'*45}")
    print(f"   Сравнение методов для x = {x}")
    print(f"{'-'*45}")
    ref = next(iter(results.values()))
    for method, val in results.items():
        print(f"   {method:<20}: {val:>12.6f}   (|Δ|={abs(val -
ref):.2e})")
    print(f"{'-'*45}")

def plot_interpolation(xs, ys, delta, x_queries=None, true_func=None,
true_fname=None):
    n_nodes = len(xs)
    h = xs[1] - xs[0]
    x_mid = (xs[0] + xs[-1]) / 2

    x_dense = np.linspace(xs[0], xs[-1], 400)

    newton_vals = []
    gauss_vals = []
    for xv in x_dense:
        if xv <= x_mid:
            newton_vals.append(newton_forward(xs, delta, xv))
        else:
            newton_vals.append(newton_backward(xs, delta, xv))

    c = n_nodes // 2
    t = (xv - xs[c]) / h
    if t >= 0:
        gauss_vals.append(gauss_forward(xs, delta, xv))
    else:
        gauss_vals.append(gauss_backward(xs, delta, xv))

    plt.style.use("seaborn-v0_8-whitegrid")
    fig, ax = plt.subplots(figsize=(10, 6))
    fig.patch.set_facecolor("#F7F9FC")
    ax.set_facecolor("#F7F9FC")

    ax.plot(x_dense, newton_vals,

```



```

        color="#1B6CA8", linewidth=2.2, linestyle="-",
        label="Многочлен Ньютона", zorder=3)

ax.plot(x_dense, gauss_vals,
        color="#E07B2A", linewidth=2.2, linestyle="--",
        label="Многочлен Гаусса", zorder=4)

if true_func is not None:
    true_vals = [true_func(xv) for xv in x_dense]
    label_true = f"Истинная функция f(x) = {true_fname}" if
true_fname else "Истинная функция"
    ax.plot(x_dense, true_vals,
            color="#2CA02C", linewidth=2.5, linestyle=(0, (4,
2)),
            label=label_true, zorder=5)

ax.scatter(xs, ys,
           color="#1B6CA8", edgecolors="#0D3B6E",
           s=80, zorder=5, label="Узлы интерполяции")

if x_queries:
    for xq in x_queries:
        if xs[0] <= xq <= xs[-1]:
            yq = lagrange(xs, ys, xq)
            ax.scatter([xq], [yq],
                       marker="*", s=220,
                       color="#D62839", edgecolors="#8B0000",
                       zorder=6, label=f"x* = {xq}")
            ax.axvline(xq, color="#D62839", linewidth=0.8,
                       linestyle=":", alpha=0.6)

ax.set_xlabel("x", fontsize=13, labelpad=8)
ax.set_ylabel("y", fontsize=13, labelpad=8)
title_suffix = f"f(x) = {true_fname}" if true_fname else
"табличные данные"
ax.set_title(f"Интерполяция функции ({title_suffix})",
             fontsize=14, fontweight="bold", pad=14)

handles, labels = ax.get_legend_handles_labels()
seen = {}
uniq_h, uniq_l = [], []
for h_, l_ in zip(handles, labels):
    if l_ not in seen:
        seen[l_] = True
        uniq_h.append(h_)
        uniq_l.append(l_)
ax.legend(uniq_h, uniq_l, fontsize=10, framealpha=0.9,
loc="best")

ax.tick_params(labelsize=10)
plt.tight_layout()
plt.show()

def get_data_source():
    valid_choices = ["1", "2", "3"]
    choice_descriptions = {
        "1": "Ввод с клавиатуры",
        "2": "Загрузка из файла",
        "3": "Встроенная функция"
    }

    while True:
        print("\n Источник данных:")
        for key in valid_choices:

```

```

        print(f"        {key}. {choice_descriptions[key]}")

    choice = input("    Выбор: ").strip()

    if not choice:
        print("Ввод не может быть пустым. Попробуйте ещё раз.")
        continue

    if choice in valid_choices:
        if choice == "1":
            xs, ys = input_manual()
            return xs, ys, None, None
        elif choice == "2":
            xs, ys = input_from_file()
            return xs, ys, None, None
        elif choice == "3":
            xs, ys, func, fname = input_from_function()
            return xs, ys, func, fname
    else:
        print("Некорректный выбор. Доступные варианты: 1,2,3")

def main():
    print_header("Интерполяция функции")
    print("Методы: Лагранж | Ньютон (конечные разности) | Гаусс")
    print("Дополнительно: Стирлинг | Бессель")

    while True:
        try:
            xs, ys, true_func, true_fname = get_data_source()
            validate_data(xs, ys)
            break
        except Exception as e:
            print(f"\n Непредвиденная ошибка: {e}")
            print("    Попробуйте ещё раз\n")

    print_table(xs, ys, "Исходные данные")

    delta = build_finite_differences(ys)
    print_finite_diff_table(xs, delta)

    print("\n    Введите точку интерполяции x* (или несколько через пробел):")
    while True:
        try:
            raw = input("    x* = ").strip()
            if not raw:
                print("Ввод не может быть пустым")
                continue
            points = [float(v) for v in raw.split()]
            break
        except ValueError:
            print("Введите корректные числа через пробел")

    for x_star in points:
        print_separator("-", 70)
        print(f"\n Интерполяция в точке x* = {x_star}")
        print_separator("-", 70)

        if x_star < xs[0] or x_star > xs[-1]:
            print("x* вне диапазона узлов — экстраполяция")

        h = xs[1] - xs[0]
        c = (len(xs) - 1) // 2
        t = (x_star - xs[c]) / h

```

```

x_mid = (xs[0] + xs[-1]) / 2
results = {}

print("\n — Метод Лагранжа")
results["Лагранж"] = lagrange(xs, ys, x_star)
print(f" L(x) = {results['Лагранж']:.6f}")

print("\n — Метод Ньютона (конечные разности)")
newton_key = "Ньютон (вперёд)" if x_star <= x_mid else
"Ньютон (назад)"
results[newton_key] = newton_finite_verbose(xs, delta,
x_star)

print("\n — Метод Гаусса")
gauss_key = "Гаусс (1-я)" if t >= 0 else "Гаусс (2-я)"
results[gauss_key] = gauss_verbose(xs, delta, x_star)

if abs(t) <= 0.25:
    print("\n — Формула Стирлинга ( $|t| \leq 0.25$ )")
    results["Стирлинг"] = stirling(xs, delta, x_star)
    print(f" S(x) = {results['Стирлинг']:.6f}")
else:
    print(f"\n [Стирлинг пропущен:  $|t|={abs(t):.3f} >$ 
0.25]")

if 0.25 <= abs(t) <= 0.75:
    print("\n — Формула Бесселя ( $0.25 \leq |t| \leq 0.75$ )")
    results["Бессель"] = bessell(xs, delta, x_star)
    print(f" B(x) = {results['Бессель']:.6f}")
else:
    print(f" [Бессель пропущен:  $|t|={abs(t):.3f}$  вне (0.25,
0.75)]")

print_comparison(results, x_star)

plot_interpolation(xs, ys, delta, x_queries=points,
true_func=true_func, true_fname=true_fname)

print(" Работа завершена")

if __name__ == "__main__":
    main()

```

Вывод программы

Методы: Лагранж | Ньютон (конечные разности) | Гаусс
Дополнительно: Стирлинг | Бессель

Источник данных:

1. Ввод с клавиатуры
2. Загрузка из файла
3. Встроенная функция

Выбор: 3

Выберите функцию:

1. $\sin(x)$
2. $\cos(x)$
3. $\exp(x)$
4. $\ln(x)$
5. x^2

Функция: 1
 Начало отрезка a: 0
 Конец отрезка b: 3
 Количество узлов (≥ 2): 7

Сгенерированы узлы для $f(x) = \sin(x)$

Исходные данные		
i	x	y
0	0.000000	0.000000
1	0.500000	0.479426
2	1.000000	0.841471
3	1.500000	0.997495
4	2.000000	0.909297
5	2.500000	0.598472
6	3.000000	0.141120

Таблица конечных разностей								
i	x	y	$\Delta^1 y$	$\Delta^2 y$	$\Delta^3 y$	$\Delta^4 y$	$\Delta^5 y$	$\Delta^6 y$
0	0.00000	0.000000	0.479426	-0.117380	-0.088641	0.050441		
0.009353	-0.014640							
1	0.50000	0.479426	0.362045	-0.206021	-0.038200	0.059794	-	
0.005287								
2	1.00000	0.841471	0.156024	-0.244222	0.021594	0.054507		
3	1.50000	0.997495	-0.088198	-0.222628	0.076101			
4	2.00000	0.909297	-0.310825	-0.146527				
5	2.50000	0.598472	-0.457352					
6	3.00000	0.141120						

Введите точку интерполяции x^* (или несколько через пробел):
 $x^* = 0.3 \ 1.6 \ 1.75 \ 2.3$

Интерполяция в точке $x^* = 0.3$

— Метод Лагранжа
 $L(x) = 0.295541$

— Метод Ньютона (конечные разности)

Ньютон (вперёд): $t = 0.6000$
 $N(x) = 0.295541$

— Метод Гаусса

Многочлен Гаусса для $x = 0.3$
Центральный узел: $x_0 = 1.50000$, $t = -2.4000$
Формула: 2-я ($t < 0$)
 $G(x) = 0.295541$

[Стирлинг пропущен: $|t|=2.400 > 0.25$]
[Бессель пропущен: $|t|=2.400$ вне $(0.25, 0.75)$]

Сравнение методов для $x = 0.3$

Лагранж	:	0.295541	($ \Delta =0.00\text{e}+00$)
Ньютон (вперёд)	:	0.295541	($ \Delta =3.89\text{e}-16$)
Гаусс (2-я)	:	0.295541	($ \Delta =4.44\text{e}-16$)

Интерполяция в точке $x^* = 1.6$

— Метод Лагранжа

$L(x) = 0.999573$

— Метод Ньютона (конечные разности)

Ньютон (назад): $t = -2.8000$

$N(x) = 0.999573$

— Метод Гаусса

Многочлен Гаусса для $x = 1.6$
Центральный узел: $x_0 = 1.50000$, $t = 0.2000$
Формула: 1-я ($t \geq 0$)
 $G(x) = 0.999573$

— Формула Стирлинга ($|t| \leq 0.25$)

$S(x) = 0.999573$

[Бессель пропущен: $|t|=0.200$ вне $(0.25, 0.75)$]

Сравнение методов для $x = 1.6$

Лагранж	:	0.999573	($ \Delta =0.00\text{e}+00$)
Ньютон (назад)	:	0.999573	($ \Delta =1.11\text{e}-16$)
Гаусс (1-я)	:	0.999573	($ \Delta =2.22\text{e}-16$)
Стирлинг	:	0.999573	($ \Delta =2.22\text{e}-16$)

Интерполяция в точке $x^* = 1.75$

— Метод Лагранжа

$L(x) = 0.983985$

— Метод Ньютона (конечные разности)

Ньютон (назад): $t = -2.5000$

$N(x) = 0.983985$

— Метод Гаусса

Многочлен Гаусса для $x = 1.75$

Центральный узел: $x_0 = 1.50000$, $t = 0.5000$

Формула: 1-я ($t \geq 0$)

$G(x) = 0.983985$

[Стирлинг пропущен: $|t|=0.500 > 0.25$]

— Формула Бесселя ($0.25 \leq |t| \leq 0.75$)

$B(x) = 0.953396$

Сравнение методов для $x = 1.75$

Лагранж : 0.983985 ($|\Delta|=0.00\text{e}+00$)

Ньютон (назад) : 0.983985 ($|\Delta|=1.11\text{e}-16$)

Гаусс (1-я) : 0.983985 ($|\Delta|=1.11\text{e}-16$)

Бессель : 0.953396 ($|\Delta|=3.06\text{e}-02$)

Интерполяция в точке $x^* = 2.3$

— Метод Лагранжа

$L(x) = 0.745704$

— Метод Ньютона (конечные разности)

Ньютон (назад): $t = -1.4000$

$N(x) = 0.745704$

— Метод Гаусса

Многочлен Гаусса для $x = 2.3$

Центральный узел: $x_0 = 1.50000$, $t = 1.6000$

Формула: 1-я ($t \geq 0$)

$G(x) = 0.745704$

[Стирлинг пропущен: $|t|=1.600 > 0.25$]

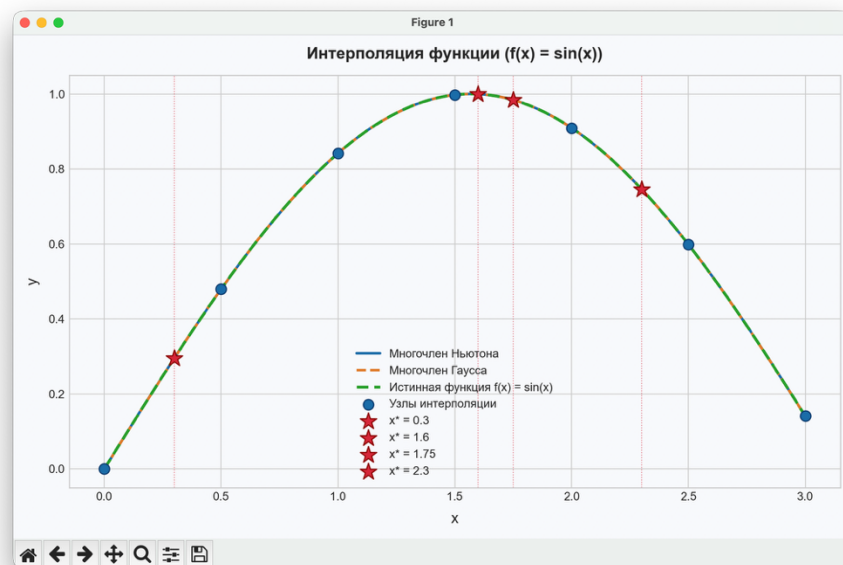
[Бессель пропущен: $|t|=1.600$ вне $(0.25, 0.75)$]

Сравнение методов для $x = 2.3$

Лагранж : 0.745704 ($|\Delta|=0.00\text{e}+00$)

Ньютон (назад) : 0.745704 ($|\Delta|=2.22\text{e-}16$)
Гаусс (1-я) : 0.745704 ($|\Delta|=2.22\text{e-}16$)

Работа завершена



Вывод

Выполнение лабораторной работы по вычислительной математике позволило мне познакомиться с интерполяцией. Данная работа дала мне возможность применить полученные теоретические знания на практике, развить навыки работы с информацией. В процессе выполнения я вручную воспользовалась первой интерполяционной формулой Ньютона, первой интерполяционной формулой Гаусса, а также многочленом Лагранжа и написала программу, реализующую вычисления приближенных значений функции для заданного значения аргумента различными методами (многочлен Лагранжа, многочлен Ньютона с конечными разностями, многочлен Гаусса), на ЯП Python. Приобретённый опыт поможет мне при дальнейшем изучении предмета вычислительная математика.